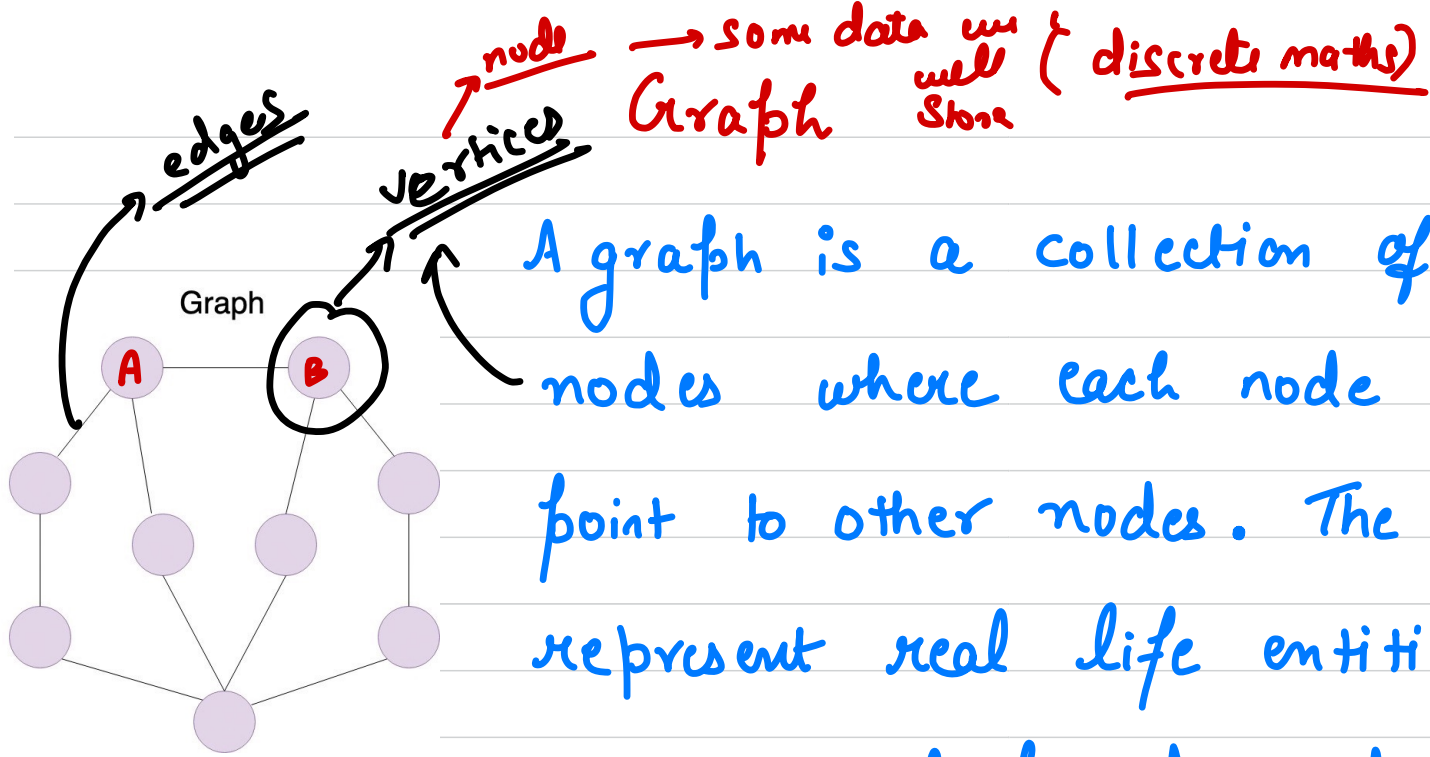


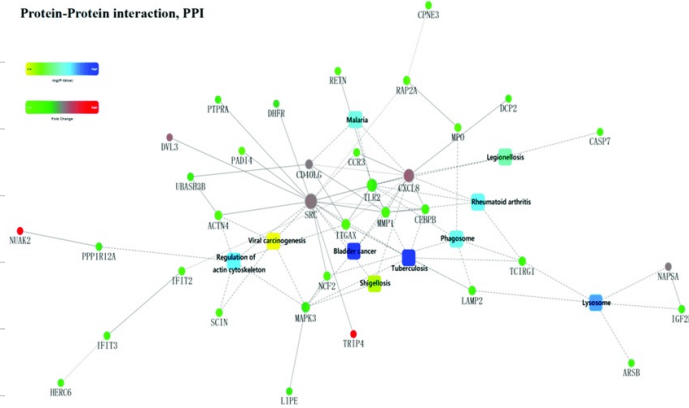


A graph is a collection of nodes where each node might point to other nodes. The nodes represent real life entities and are connected by edges representing relationship between the entities/node.

(Social media)

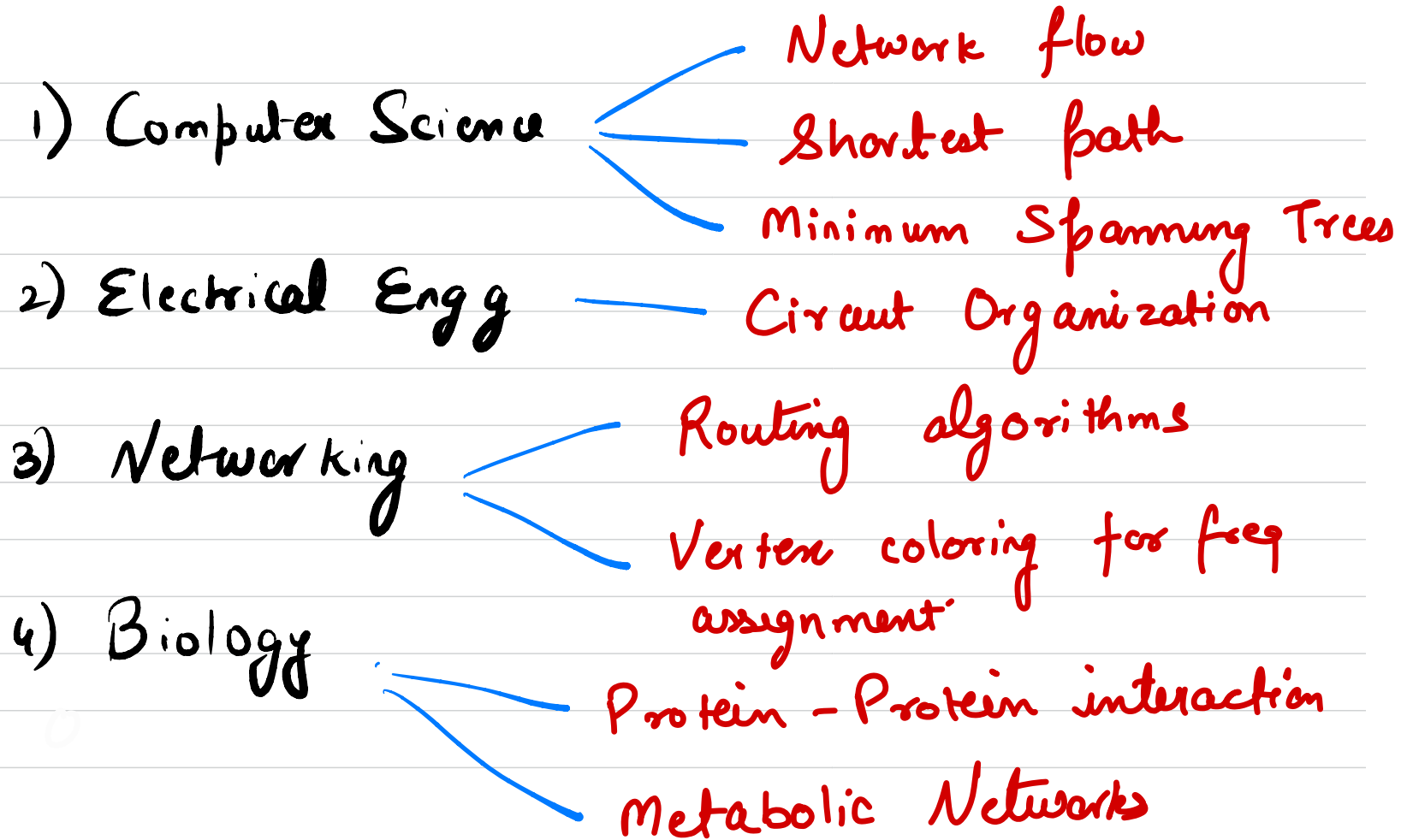
Applications Of Graphs

Protein-Protein interaction, PPI



Protein - Protein
Interaction

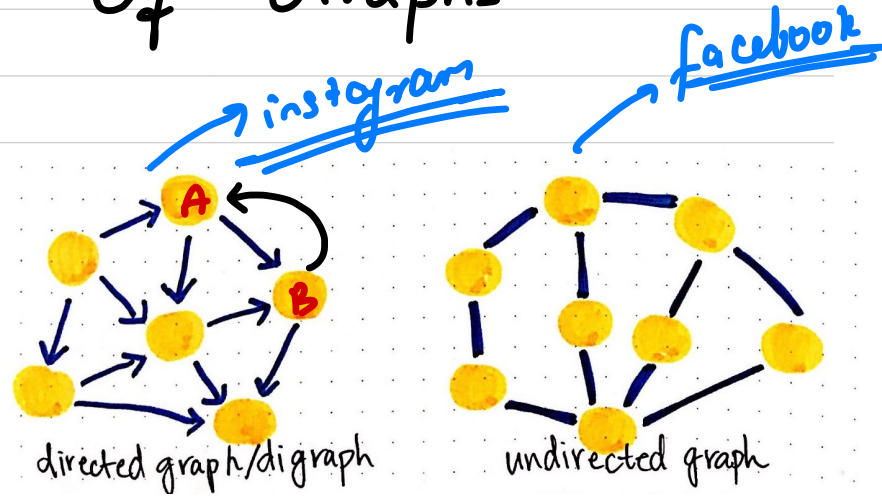
Google Maps



Type Of Graphs

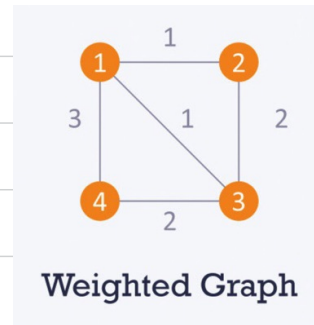
1) Undirected Graph

2) Directed Graph

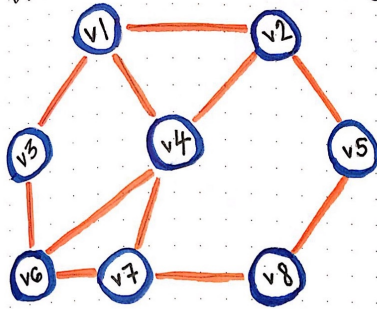


3) Weighted Graph

4) Unweighted graphs



(Formally) Defining a Graph



8 vertices/nodes

11 edges/links

$$G = (\underbrace{V, E}_{\text{pair}})$$

$$V = \{v1, v2, v3, v4, v5, v6, v7, v8\}$$

$$E = \{ \{v1, v2\}, \{v1, v3\}, \{v1, v4\}, \{v2, v4\}, \{v2, v5\}, \{v3, v6\}, \{v4, v6\}, \{v4, v7\}, \{v5, v8\}, \{v6, v7\}, \{v7, v8\} \}$$

~~set~~

set of pairs

these edge definitions are unordered pairs!

→ $G = (V, E)$ is the formal mathematical notation for defining graphs.

→ A graph G is an ordered pair of a set V vertices and E , a set of edges.

→ An ordered pair is a pair of mathematical objects in which the order of objects in the pair matters.



But what about a directed graph?

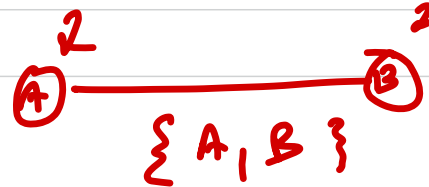
$$G = (V, E) \quad \text{how would our edge objects be different?}$$

$$V = \{ v1, v2, v3 \}$$

$$E = \{ (v1, v2), (v1, v3), (v2, v3) \}$$

pair ordering matters

these edge definitions are ordered pairs, because direction matters!



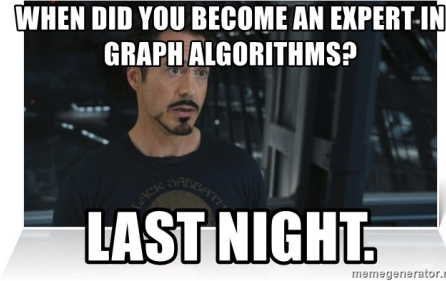
Representation Of Graphs

1) Adjacency List $\rightarrow$ array of LL

2) Adjacency Matrix $\rightarrow$ 2D array

3) Adjacency Map $\rightarrow$ array of hashmap /
hash map of nodes

4) Incidence Matrix $\rightarrow$ 2D array



Vertices

[C][1]
C

Adjacency List

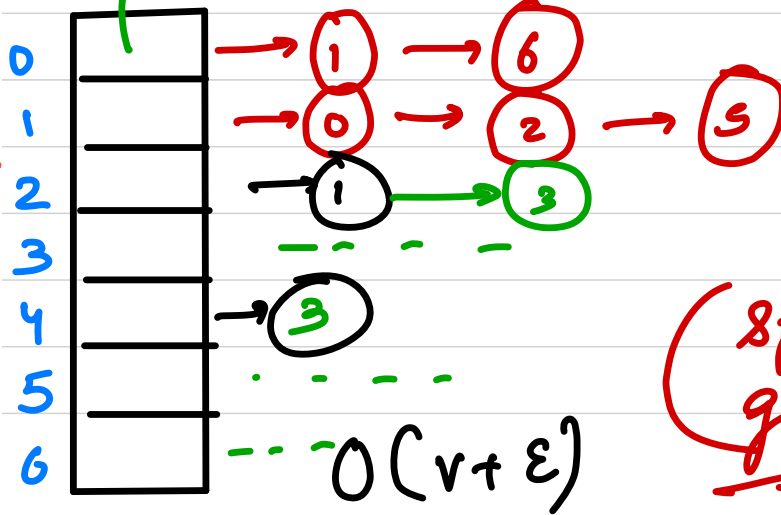
→ sparse ok
→ add at head $O(1)$

Array of linked list.

we store immediate
neighbour of ith node

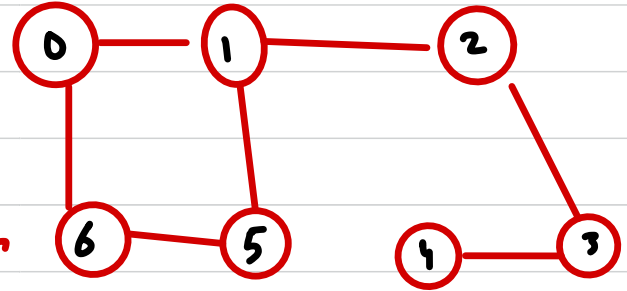
edges

(vertices, wt)



V E

$2G$



sparse graph

→ hashmap

→ list
→ bucket

Adjacency Matrix

simple

2d array

$$A_{ij} = \begin{cases} 1 \\ 0 \end{cases}$$

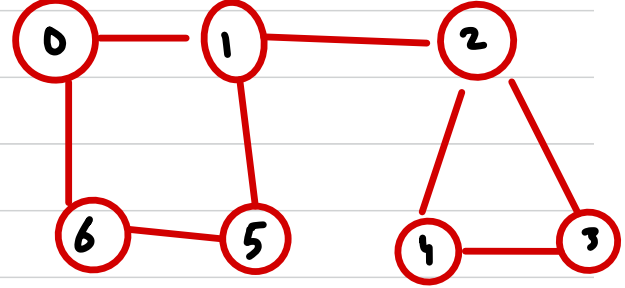
there is an
edge between i & j
else

Dense graphs

2G

A =

	0	1	2	3	4	5	6
0	0	1	0	0	0	0	1
1	1	0	1	0	0	1	0
2	0	1	0	1	1	0	0
3	0	0	1	0	1	0	0
4	0	0	1	1	0	0	0
5	0	1	0	0	0	0	1
6	1	0	0	0	0	1	



$O(V^2)$

$V \times V$

$V_1 - V_2$

$m[V_1][V_2] == 1$

2D matrix

Incidence Matrix

$m_{ij} = \begin{cases} 1 & \text{if } i^{\text{th}} \text{ vertex belongs to } j^{\text{th}} \text{ edge} \\ 0 & \text{else} \end{cases}$

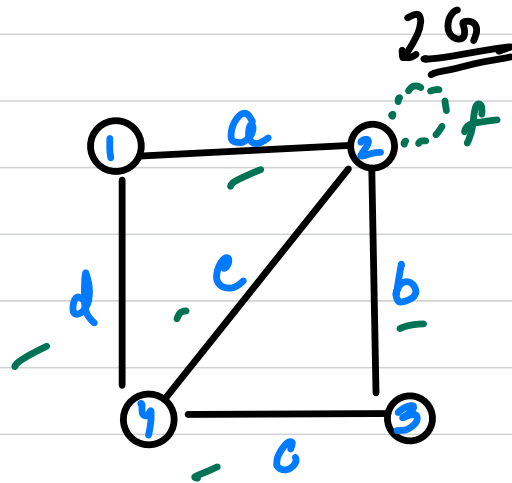
$M =$

	a	b	c	d	e	f
1	1	0	0	1	0	
2	1	1	0	0	1	
3	0	1	1	0	0	
4	0	0	1	1	1	

column represent edges

rows represent vertices

row sum = degree
column sum = 2



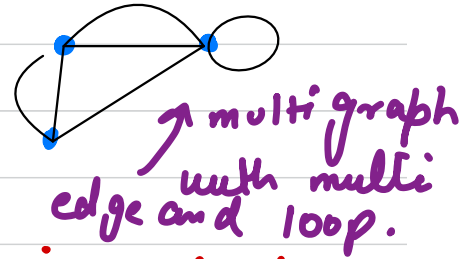
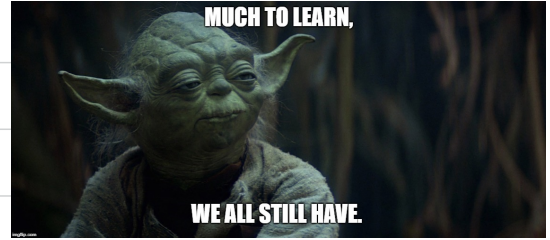
Let's Implement Graphs

- Vectors / Array list
- Linked List
- Structs / Classes



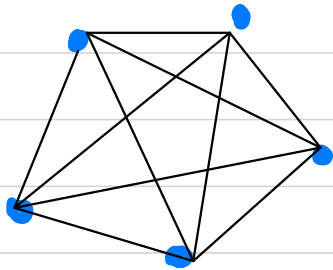
Graph Terminologies

Multi Graph $\rightarrow$ An undirected graph in which multiple edges are allowed between any 2 vertices.



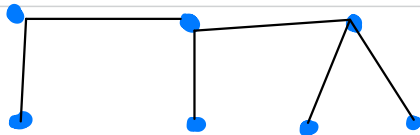
Simple Graph $\rightarrow$ An undirected graph in which both multiple edges and loops are not allowed.

Complete Graph $\rightarrow$ A graph in which every vertex is directly connected to every vertex.



$\swarrow$ example of complete graph.

Connected Graph $\rightarrow$ A connected graph has a path from every vertex to all other vertex.



$\swarrow$ connected graph.

Path $\rightarrow$ A path P_n is a graph whose vertices can be arranged in a sequence.

$v_1 \quad v_2 \quad v_3 \quad v_n$
 $V = \{v_1, v_2, v_3, \dots, v_n\}$ Such
 $\hookrightarrow$ Path that edge set of graph is

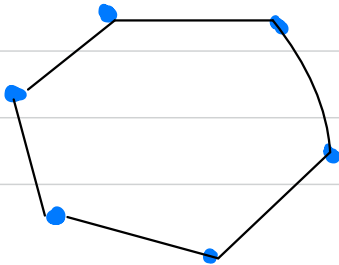
$$E = \{v_i v_{i+1} \quad \forall i \in [1, n-1]\}$$

Cycle $\rightarrow$ A cycle C_n is a graph whose vertices can be arranged in a cyclic sequence

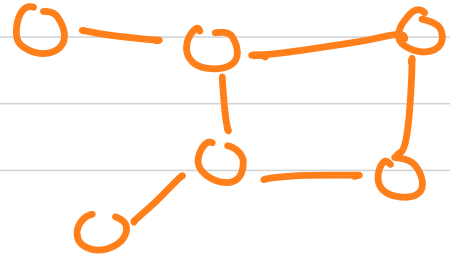
$V = \{v_1, v_2, v_3, \dots, v_n\}$ such that the

edge set is

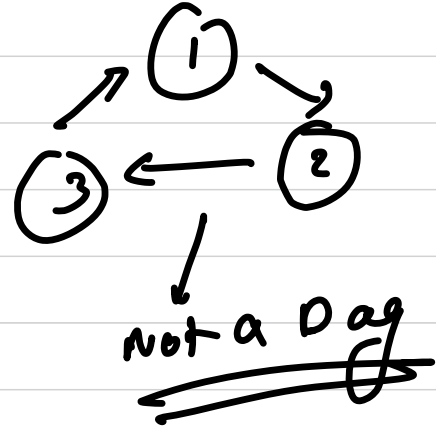
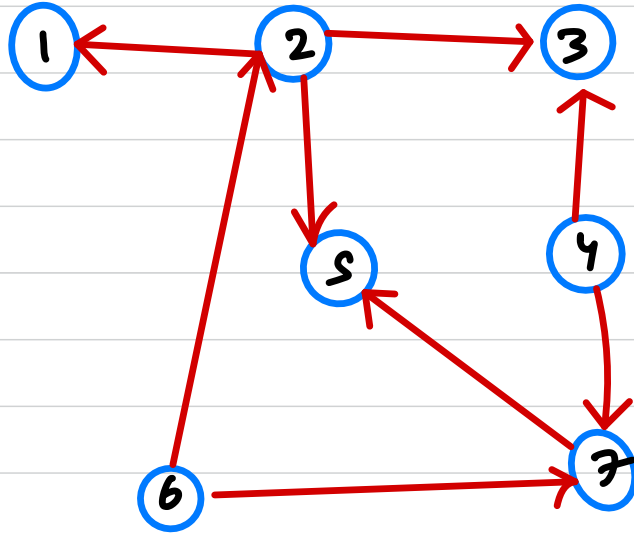
$$E = \{v_i v_{i+1} \mid i \in [1, n-1]\} \cup \{v_1 v_n\}$$



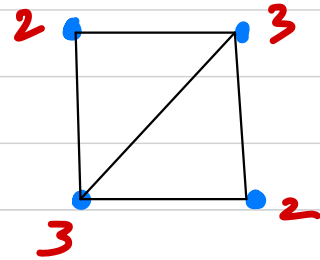
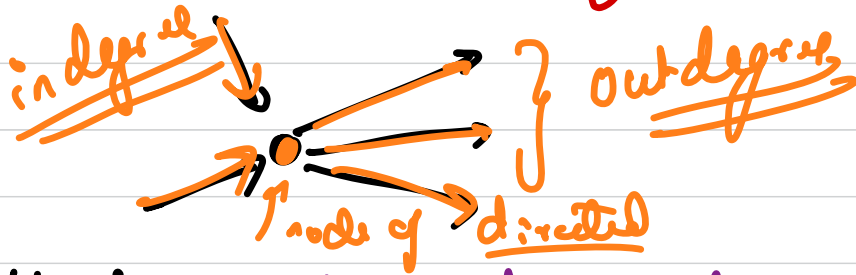
$\swarrow$ cycle



DAG (Directed Acyclic Graph)



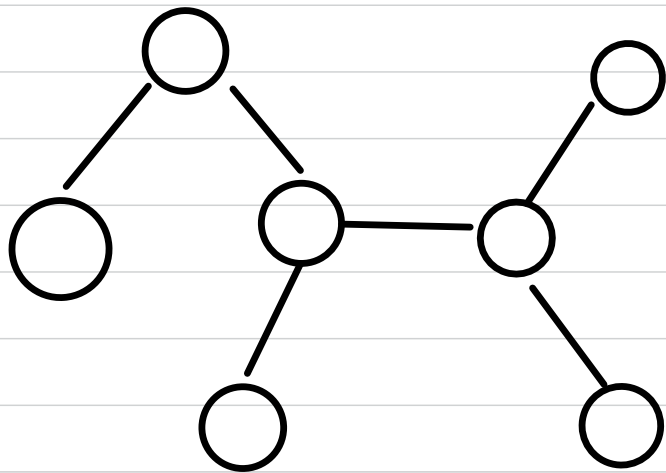
Degree $\rightarrow$ Degree of a vertex in a graph is the total no. of edges incident to it/ associated with it (for undirected graph)



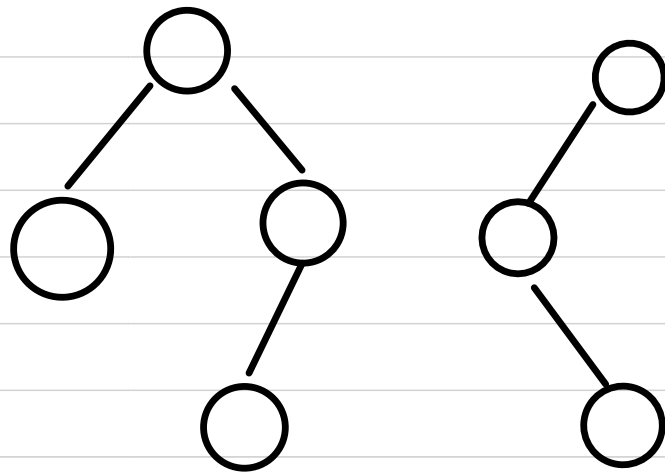
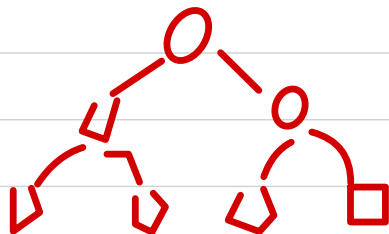
Note $\rightarrow$ In a directed graph, the out degree of a vertex is total no. of outgoing edges & indegree is total no. of incoming edges.

Tree $\rightarrow$ It is a connected graph with no cycles.
If we remove all cycles from a graph we will get a tree. And if we remove an edge from a tree, it is called forest.

forest $\rightarrow$ An undirected graph where any 2 vertices are connected by at most 1 path.



Tree



forest

Relationship between number of edges & vertices

Directed $|E| = |V| \times (|V| - 1)$ (max)

Undirected $|E| = |V|C_2$ (max)

Connected $|E| = |V| - 1$ (min) nC_1

Tree $|E| = |V| - 1$

Forest $|E| = |V| - 1$

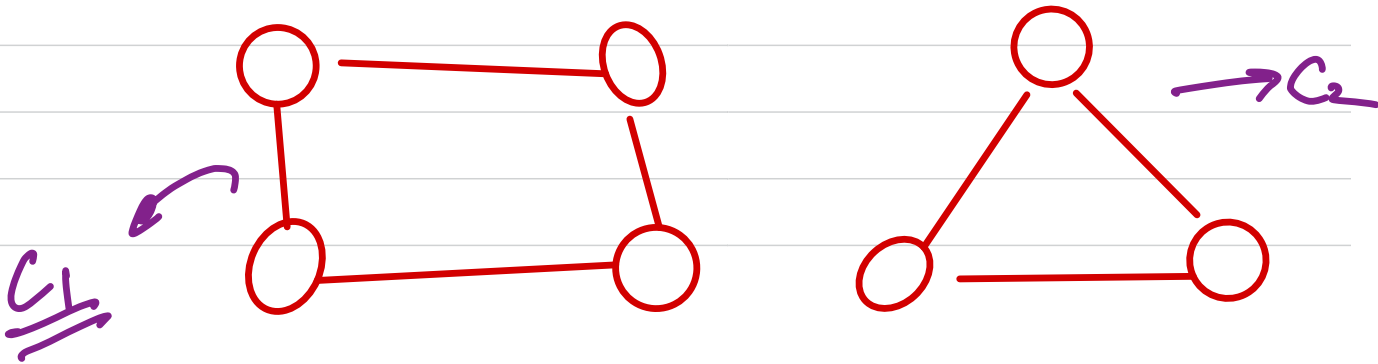
Complete $|E| = |V|C_2$

$$|V|C_2$$

max

Connected Components

* Component. $\rightarrow$ If there is a dis-connected graph, then the set of vertices which are connected forms a component.



How To Read Graphs

As graphs are non-linear data structure, we need some mechanism to read graphs. These mechanisms are also called as Graph Traversals.

① Depth First Search (DFS) ↗ Recursive

② Breadth First Search (BFS) ↘ Iterative



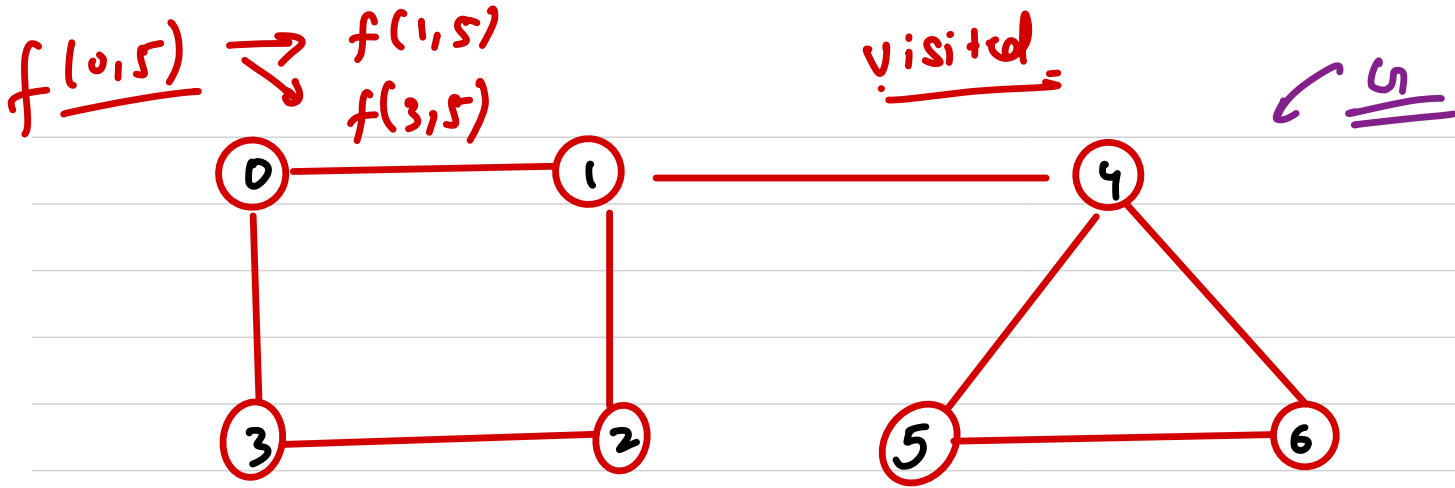
Depth first Search

Lets take a motivation problem :

Given a graph calculate all paths between 2 vertices

OR

Given a graph check whether there is a path between any 2 vertices.



0-5 (any path from 0 to 5) ✓

0-1 $\leftarrow$ direct edge
0-3 $\leftarrow$

$f(u, v)$



whether there is a
path from u to v

or not

=



$f(n_1, v)$

or

$f(n_2, v)$

or

$f(n_3, v)$

or

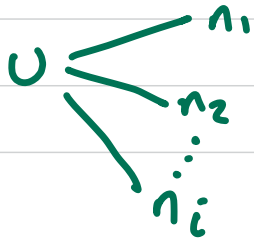
$\vdots$

$f(n_i, v)$

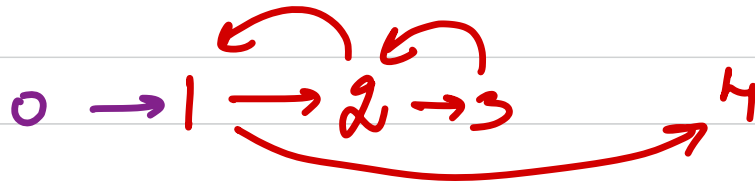
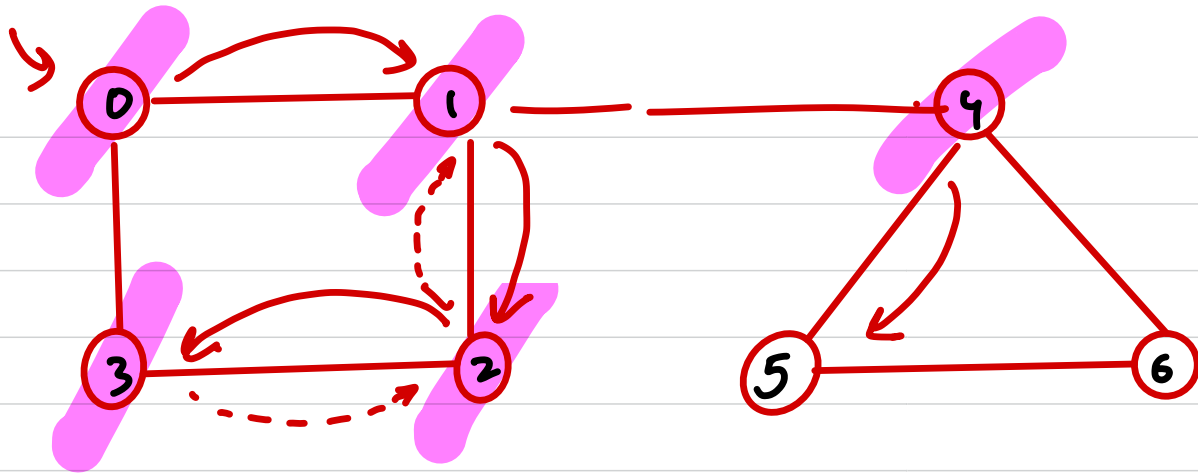
or

Recursion

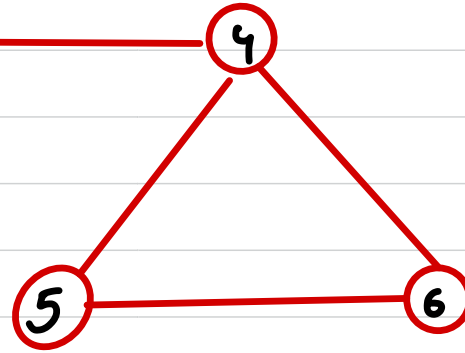
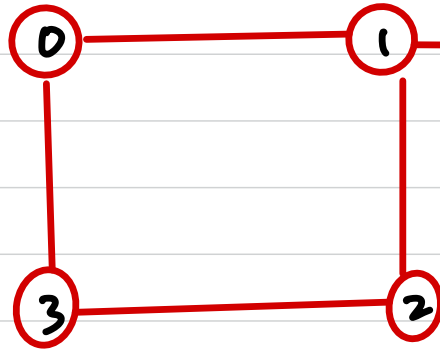
[]



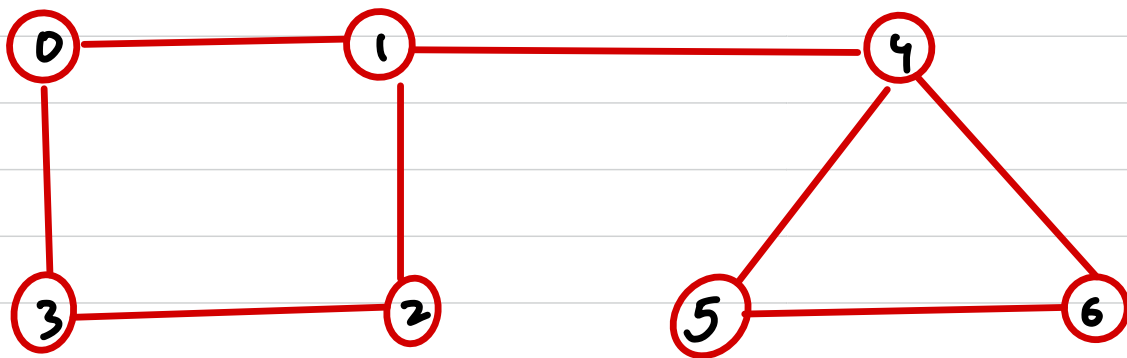
$n_1, n_2 \dots n_i$ are immediate neighbors
of u .



boolean []
hashmap } → visited



Rekursiv
Stack



$g(0,5)$

0, 1, 4, 5

0, 1, 4, 6, 5

0, 3, 2, 1, 4, 5

0, 3, 2, 1, 4, 6, 5

$$g(u, v) = u + g(n_j, v)$$

all paths from

$u \rightarrow v$

$j \in [0, i]$

$n_i \rightarrow$ neighbour of v



```

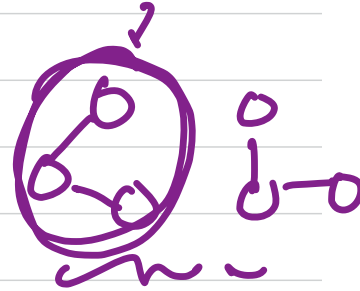
57 bool has_path(std::vector<std::list<int> > &graph, int start, int end, std::vector<bool> &
58
59     if(start == end) return true;
60     visited[start] = true;
61     for(auto &neighbour : graph[start]) {
62         if(not visited[neighbour]) {
63             bool result = has_path(graph, neighbour, end, visited);
64             if(result) return true;
65         }
66     }
67     return false;
68 }
69
70 bool validPath(int n, std::vector<std::vector<int>>& edges, int start, int end) {
71     std::vector<std::list<int> > graph(n, std::list<int> ());
72     for(auto &edge : edges) {
73         int u = edge[0];
74         int v = edge[1];
75         graph[u].push_back(v);
76         graph[v].push_back(u);
77     }
78     std::vector<bool> visited(n, false);
79     return has_path(graph, start, end, visited);
80 }
81

```

Recursive

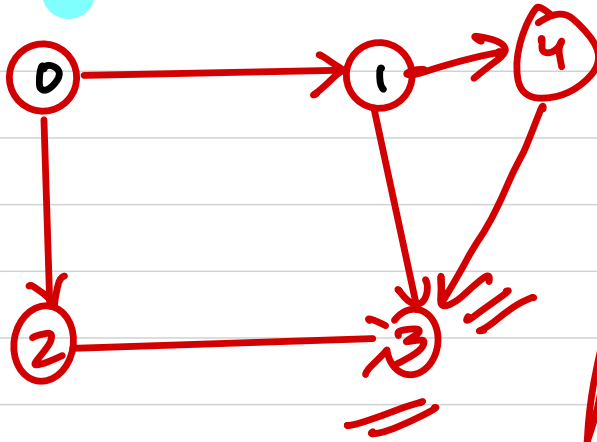
Dfs

Call stack



Time $\rightarrow O(V + E)$

DFS



visited
[0, 2, 3]

{0, 2, 3}

xy
3
path

{

{0, 1, 3}

{0, 1, 4, 3}

{0, 2, 3}

scout

copy

}

```

57 std::vector<std::vector<int> > result;
58
59 void all_path(std::vector<std::vector<int>>& graph, int start, int end, std::vector<int> &path)
60 {
61     if(start == end) {
62         path.push_back(end);
63         result.push_back(path);
64         path.pop_back();
65         return;
66     }
67     path.push_back(start);
68     visited[start] = true;
69     for(auto &neighbour : graph[start]) {
70         if(not visited[neighbour]) {
71             all_path(graph, neighbour, end, path, visited);
72         }
73     }
74     visited[start] = false;
75     path.pop_back();
76 }
77
78
79 std::vector<std::vector<int>> allPathsSourceTarget(std::vector<std::vector<int>>& graph) {
80     std::vector<int> path;
81     int n = graph.size();
82     std::vector<bool> visited(n, false);
83     all_path(graph, 0, n-1, path, visited);
84     return result;
85 }

```

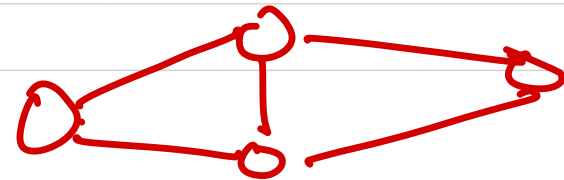
space $O(V)$ start end

$V-2$

$O(V!)$

space $O(V^3)$

converted
complete graphs

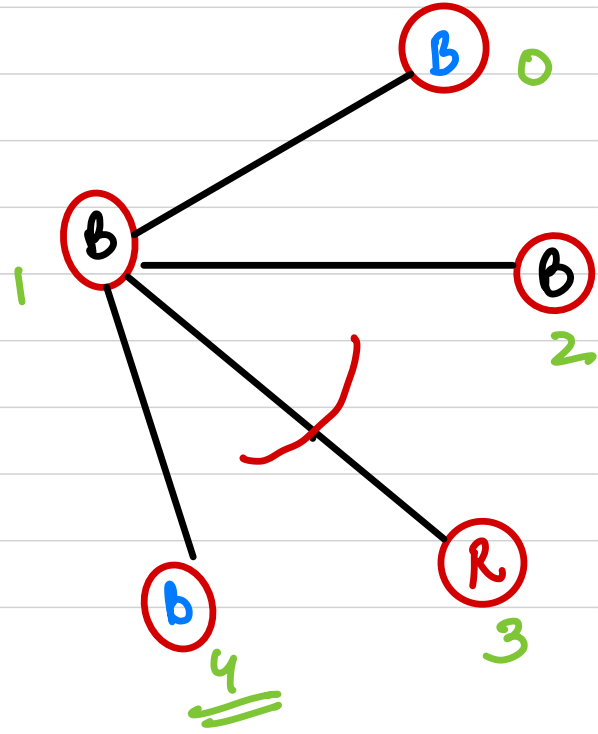


TREE CUTTING - Codeforces (Div 3-F.) (1800)

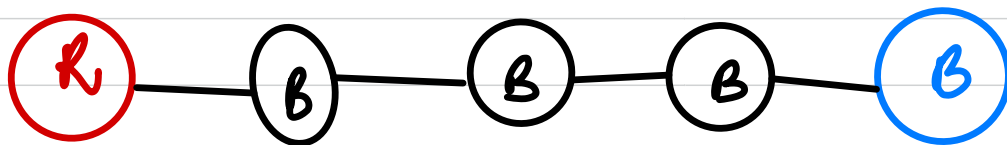
R ✓
Blue ✓
R-black ✓
b-black ✓
R-black

1
1
1

Example



Example



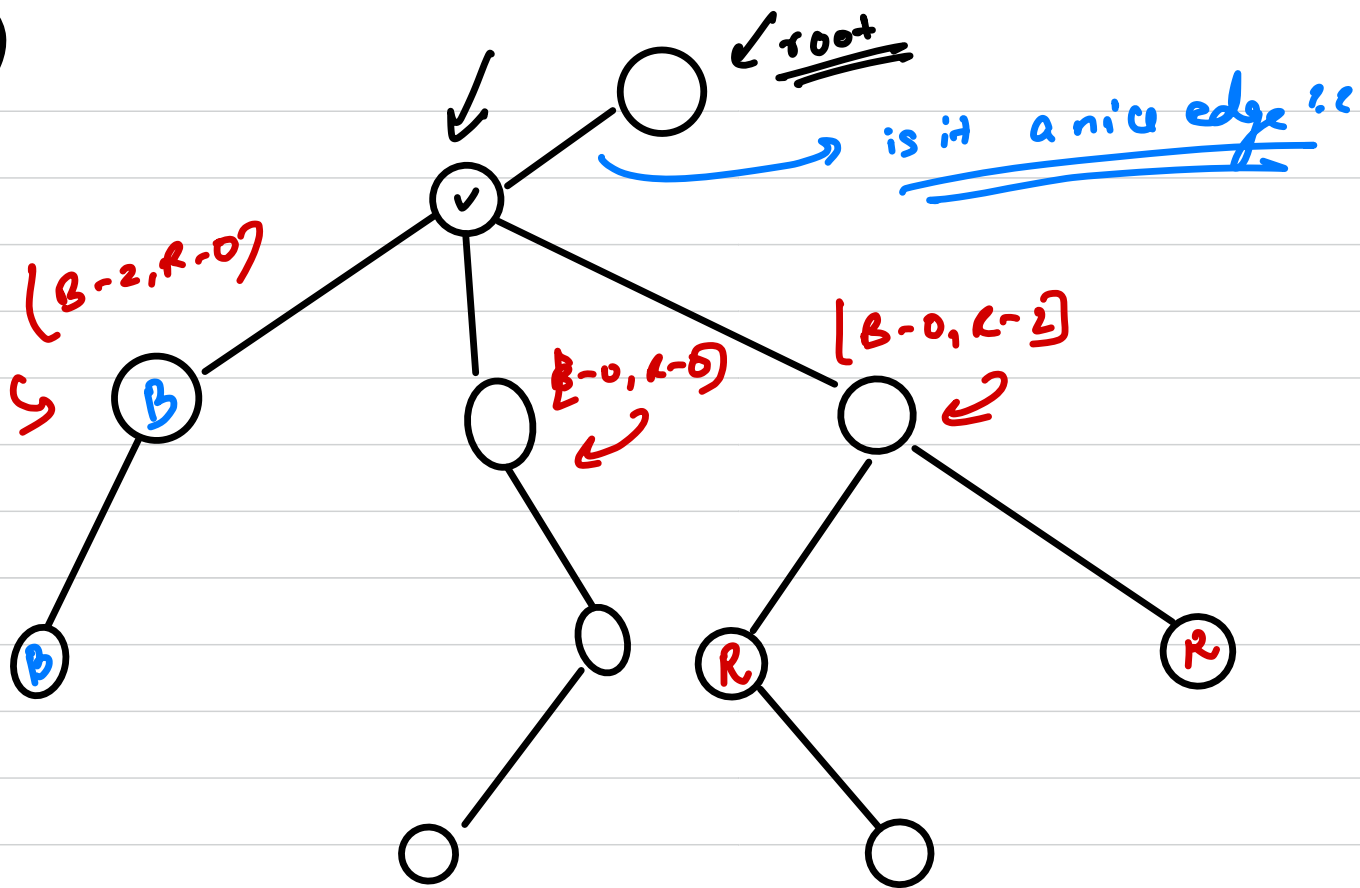
4

→ Tree → no cycles

T_B, T_R

off

Pair



we need to get the distribution of **Red**
and **Blue** nodes in a subtree

$\langle R, B \rangle$

$\boxed{R > 0 \text{ and } B > 0} \rightarrow \text{then the}$
 $\text{edge is not } \underline{\underline{\text{nice}}}$

$f(v, c)$

=

$\sum f(u, c)$

+ $(v.\text{color} == c)$

$u \in \text{neighbors of } v$

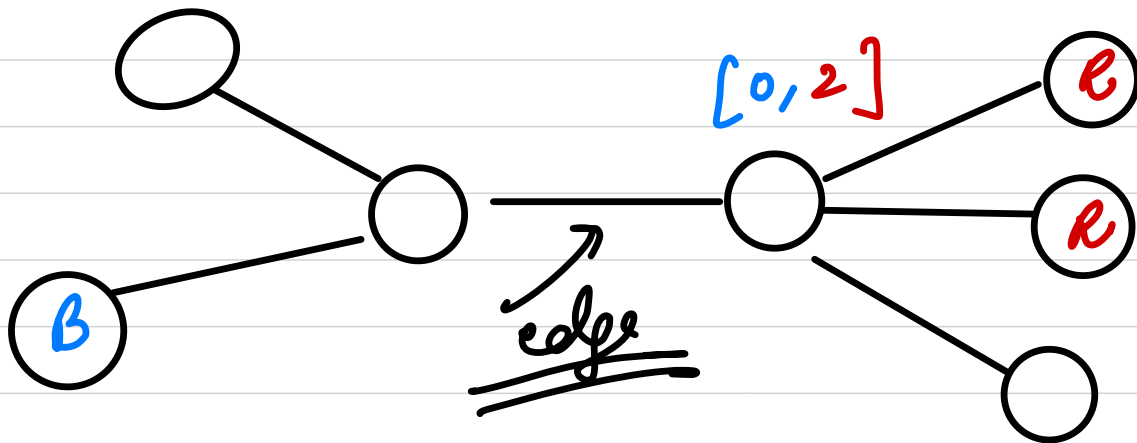
returns the

distributed of

blue & red nodes

for the tree rooted

at v



T_{R-2}
 T_{B-1}

Problem: Prove that a tree with n nodes has $n-1$ edges.

Principal of Mathematical Induction $\Rightarrow$ (P.M.I)

$\rightarrow f(n)$
 $\swarrow$

This funcⁿ returns the
no. of edges in a tree with
 n nodes

1) Base Case $n=1$

$$f(1) \rightarrow \underline{\underline{0}}$$

(Proved for base case)

2) Assumption

$$f(n) \rightarrow n-1$$

(we assume that $f(n)$
works completely fine)

3) Proof (Self work)

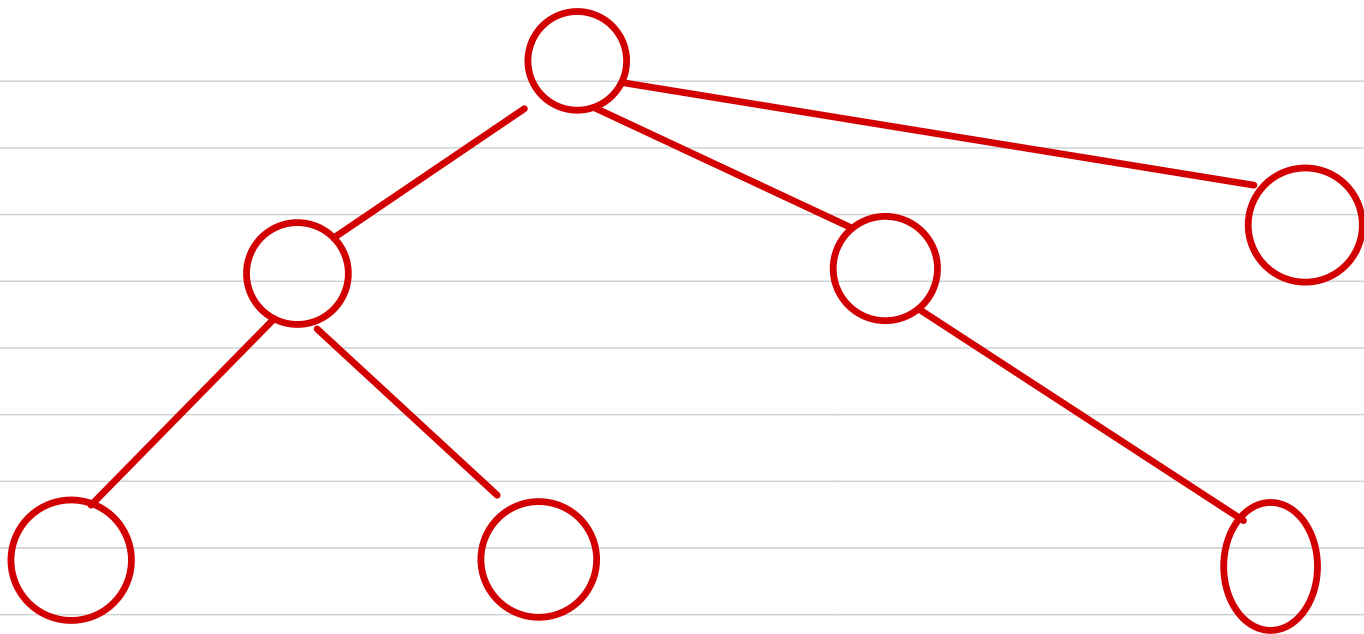
We will try to calc the value of

$$f(n+1)$$

$(n+1) \rightarrow$ tree is having $n+1$ nodes

$$f(n) = n-1$$

$n+1$ node tree is equal to a tree with n nodes & one more node getting attached to the tree.



$$f(n+1) = f(n) + \text{no. of edges reqd to add}$$

one more node to the tree

$$f(n+1) = (n-1) + 1$$

$$\boxed{f(n+1) = n}$$

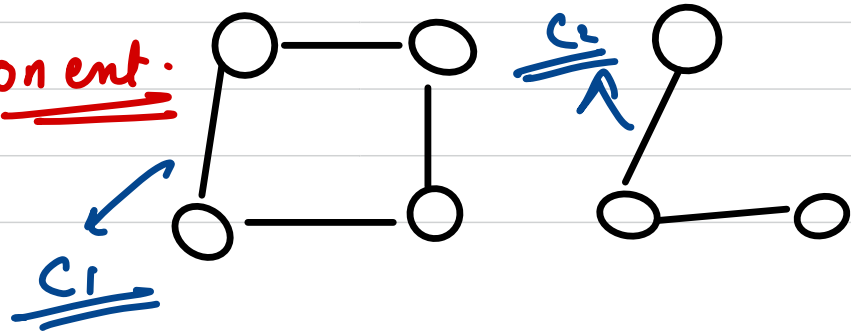
H.P

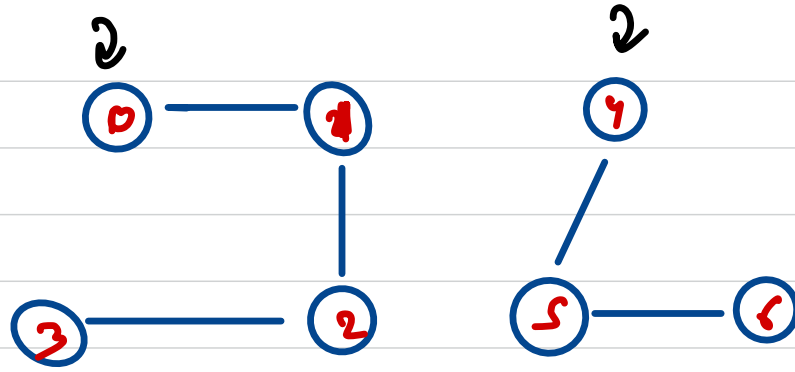
What is Connected Component ??

In a graph a set of vertices connected with each other forms one connected component.

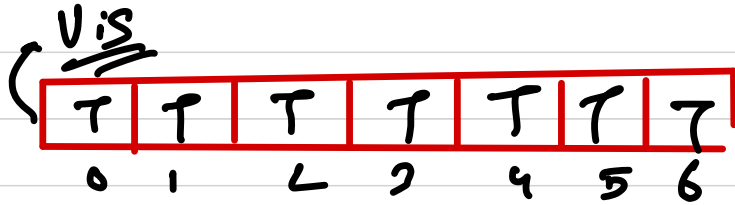
If the graph is disconnected then it will be having more than one connected component.

else only one component.





CC = 2



algo $\rightarrow$ The no of times we start doing dfs or bfs
 will be the no. of CC.

Clone A graph

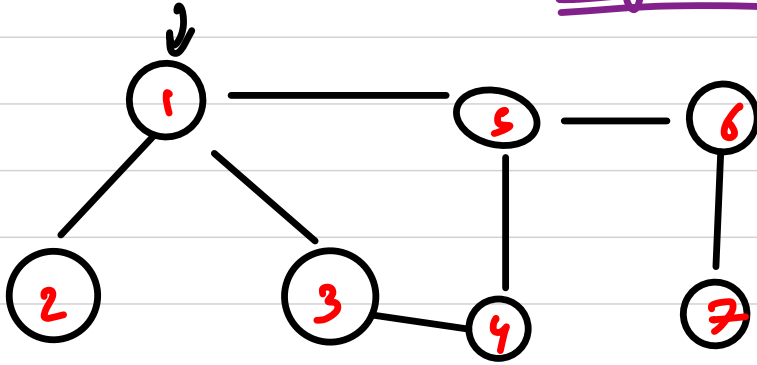
→ given → one single node.

→ To create the clone, we should read the graph at least once.

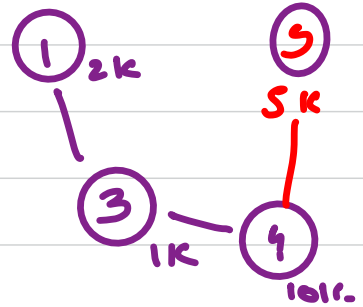
→ DFS
→ BFS

→ $O(V+E)$

adj list $\rightarrow$ new cloned graph



Original graph



[1-2k, 3-1k, 4-101k, 5-5k]

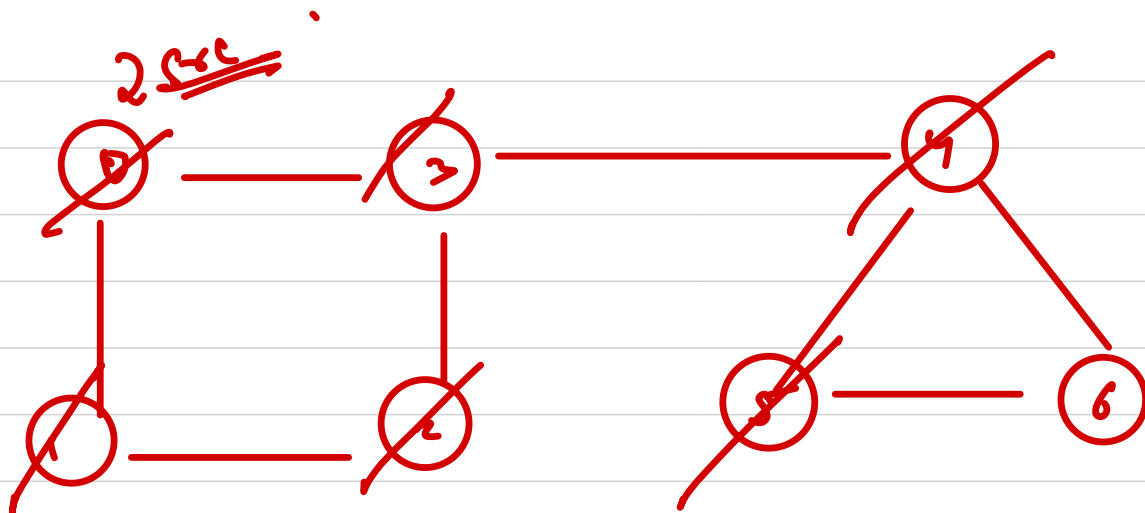
array / hash map

Start reading the graph from node 1 using dfs

prepare a visited to keep track of nodes

$$nC_r = \frac{n!}{r!(n-r)!} = \frac{nPr}{r!}$$

Visited
↓
0, 1,
3, 2, 1
4
3



← queue

